

<h1 id="Enterprise-Analytics-Dashboard">Enterprise Analytics Dashboard</h1> <p>This notebook connects to the customerstore_db Postgres database and reads directly from the analytics schema built in Stages 1 through 4 of the SQL pipeline. No raw tables are queried here, everything comes from the pre-built views and KPI tables.</p>

<h2 id="1.-Setup-and-Connection">1. Setup and Connection</h2>

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from sqlalchemy import create_engine

sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")
```

```
In [2]: pip install pandas numpy matplotlib seaborn sqlalchemy psycopg2-binary
```

```

Collecting pandas
  Downloading pandas-3.0.3-cp314-cp314-win_amd64.whl.metadata (19 kB)
Collecting numpy
  Downloading numpy-2.5.1-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Collecting matplotlib
  Downloading matplotlib-3.11.0-cp314-cp314-win_amd64.whl.metadata (80 kB)
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Collecting sqlalchemy
  Downloading sqlalchemy-2.0.51-cp314-cp314-win_amd64.whl.metadata (9.8 kB)
Collecting psycog2-binary
  Downloading psycog2_binary-2.9.12-cp314-cp314-win_amd64.whl.metadata (5.1 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\drsho\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\drsho\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from pandas) (2026.2)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp314-cp314-win_amd64.whl.metadata (5.5 kB)
Collecting cycycler>=0.10 (from matplotlib)
  Downloading cycycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.63.0-cp314-cp314-win_amd64.whl.metadata (121 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.0-cp314-cp314-win_amd64.whl.metadata (5.2 kB)
Requirement already satisfied: packaging>=20.0 in C:\Users\drsho\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from matplotlib) (26.2)
Collecting pillow>=9 (from matplotlib)
  Downloading pillow-12.3.0-cp314-cp314-win_amd64.whl.metadata (9.3 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Collecting greenlet>=1 (from sqlalchemy)
  Downloading greenlet-3.5.3-cp314-cp314-win_amd64.whl.metadata (3.9 kB)
Requirement already satisfied: typing-extensions>=4.6.0 in C:\Users\drsho\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from sqlalchemy) (4.16.0)
Requirement already satisfied: six>=1.5 in C:\Users\drsho\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading pandas-3.0.3-cp314-cp314-win_amd64.whl (9.9 MB)
----- 0.0/9.9 MB ? eta -:-:-
- ----- 0.3/9.9 MB ? eta -:-:-
--- ----- 0.8/9.9 MB 3.0 MB/s eta 0:00:03
----- 2.4/9.9 MB 5.0 MB/s eta 0:00:02
----- 5.2/9.9 MB 7.8 MB/s eta 0:00:01
----- 8.9/9.9 MB 10.4 MB/s eta 0:00:01
----- 9.9/9.9 MB 9.9 MB/s 0:00:01
Downloading numpy-2.5.1-cp314-cp314-win_amd64.whl (12.6 MB)
----- 0.0/12.6 MB ? eta -:-:-
----- 3.4/12.6 MB 17.1 MB/s eta 0:00:01
----- 4.2/12.6 MB 17.4 MB/s eta 0:00:01
----- 4.2/12.6 MB 17.4 MB/s eta 0:00:01
----- 5.8/12.6 MB 7.9 MB/s eta 0:00:01
----- 12.6/12.6 MB 12.8 MB/s 0:00:00
Downloading matplotlib-3.11.0-cp314-cp314-win_amd64.whl (9.5 MB)
----- 0.0/9.5 MB ? eta -:-:-
----- 2.4/9.5 MB 12.1 MB/s eta 0:00:01
----- 3.7/9.5 MB 12.3 MB/s eta 0:00:01
----- 3.7/9.5 MB 12.3 MB/s eta 0:00:01
----- 5.2/9.5 MB 6.4 MB/s eta 0:00:01
----- 9.4/9.5 MB 9.4 MB/s eta 0:00:01
----- 9.5/9.5 MB 9.2 MB/s 0:00:01
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)

```

[illegible]

[illegible]

[illegible]

6/19

[illegible]

Successfully installed contourpy-1.3.3 cycycler-0.12.1 fonttools-4.63.0 greenlet-3.5.3 kiwisolver-1.5.0 matplotlib-3.11.0 numpy-2.5.1 pandas-3.0.3 pillow-12.3.0 psy-copg2-binary-2.9.12 pyparsing-3.3.2 seaborn-0.13.2 sqlalchemy-2.0.51
Note: you may need to restart the kernel to use updated packages.

WARNING: The scripts f2py.exe and numpy-config.exe are installed in 'C:\Users\d
rsho\AppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.

Consider adding this directory to PATH or, if you prefer to suppress this warning, use `--no-warn-script-location`.

```
WARNING: The scripts fonttools.exe, pyftmerge.exe, pyftsubset.exe and ttx.exe are installed in 'C:\Users\drsho\AppData\Local\Python\pythoncore-3.14-64\Scripts' which is not on PATH.
```

Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

```
In [4]: # Fill in your connection details below.
# If your postgres user does not need a password (trust auth, same as pgAdmin
# connecting without a prompt), Leave DB_PASSWORD as an empty string.

DB_USER = "postgres"
DB_PASSWORD = "mm1234"          # fill in if your connection requires one
DB_HOST = "localhost"
DB_PORT = "5432"
DB_NAME = "customerstore_db"

conn_str = f"postgresql+psycopg2://{DB_USER}:{DB_PASSWORD}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
engine = create_engine(conn_str)

# quick connection check
with engine.connect() as conn:
    print("Connected successfully.")
```

Connected successfully.

2. Load KPI and Analytics Tables

```
In [5]: exec_summary = pd.read_sql("SELECT * FROM analytics.kpi_executive_summary", engine)
monthly_revenue = pd.read_sql("SELECT * FROM analytics.monthly_revenue ORDER BY date", engine)
quarterly_revenue = pd.read_sql("SELECT * FROM analytics.quarterly_revenue ORDER BY quarter", engine)
top_bottom_products = pd.read_sql("SELECT * FROM analytics.kpi_top_bottom_products", engine)
category_performance = pd.read_sql("SELECT * FROM analytics.category_performance", engine)
customer_segments = pd.read_sql("SELECT segment, COUNT(*) AS customer_count FROM analytics.customer_segments", engine)
repeat_customers = pd.read_sql("SELECT * FROM analytics.repeat_customers", engine)
territory_rankings = pd.read_sql("SELECT * FROM analytics.kpi_territory_rankings", engine)
employee_performance = pd.read_sql("SELECT * FROM analytics.kpi_employee_performance", engine)
inventory_health = pd.read_sql("SELECT * FROM analytics.kpi_inventory_health", engine)
supplier_performance = pd.read_sql("SELECT * FROM analytics.kpi_supplier_performance", engine)
purchasing_trends = pd.read_sql("SELECT * FROM analytics.kpi_purchasing_trends", engine)

print("All tables loaded.")
```

All tables loaded.

3. Executive Summary

```
In [6]: exec_summary.T.rename(columns={0: "value"})
```

```
Out[6]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe
tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead>
<th></th> <th>value</th> </thead> <tbody> <th>total_revenue</th>
<th>total_profit</th> <th>avg_order_value</th> <th>total_purchasing_customers</th>
<th>total_orders</th> <th>repeat_rate_pct</th> <th>top_category</th>
<th>top_territory</th> <th>total_employees</th> <th>total_purchasing_spend</th>
<th>products_needing_attention</th> <th>report_generated_at</th> </tbody>
<tbody></tbody>
```

109,846,380.90

9,371,903.13

3,491.07

19119

31,465.00

39.07

Bikes

Southwest

290

63,791,994.84

373

2026-07-10 08:06:32.890582+00:00

This single row is the top line of the whole pipeline. Every number in it traces back to a Stage 1 to 4 object, nothing is recalculated from raw tables here.

Known reference points already validated during the SQL build:

- Repeat rate came out at 39.07 percent, confirmed against the segmentation counts.
- Top category is Bikes and top territory is Southwest, both expected results for this dataset.
- Products needing attention (373) is mostly raw material and component rows with no retail category, plus a large share of Bikes driven by tight safety stock margins relative to other categories.

```
In [22]: kpi_labels = ["Total Revenue", "Total Profit", "Total Orders", "Repeat Rate %",
kpi_values = [
    exec_summary["total_revenue"][0],
    exec_summary["total_profit"][0],
    exec_summary["total_orders"][0],
    exec_summary["repeat_rate_pct"][0],
    exec_summary["total_employees"][0]
]

fig, axes = plt.subplots(1, len(kpi_labels), figsize=(16, 4))
for ax, label, value in zip(axes, kpi_labels, kpi_values):
    ax.text(0.5, 0.5, f"{value:,.1f}", ha="center", va="center", fontsize=20, fontweight="bold")
    ax.set_title(label, fontsize=11)
```

```
ax.axis("off")

plt.suptitle("Executive KPI Summary", fontsize=14)
plt.tight_layout()
plt.show()
```

Executive KPI Summary				
Total Revenue	Total Profit	Total Orders	Repeat Rate %	Total Employees
109,846,380.9	9,371,903.1	31,465.0	39.1	290.0

4. Monthly Revenue Trend

```
In [23]: # Section 4, revenue trend
print(monthly_revenue[["order_month", "revenue", "mom_growth_pct"]].describe())
```

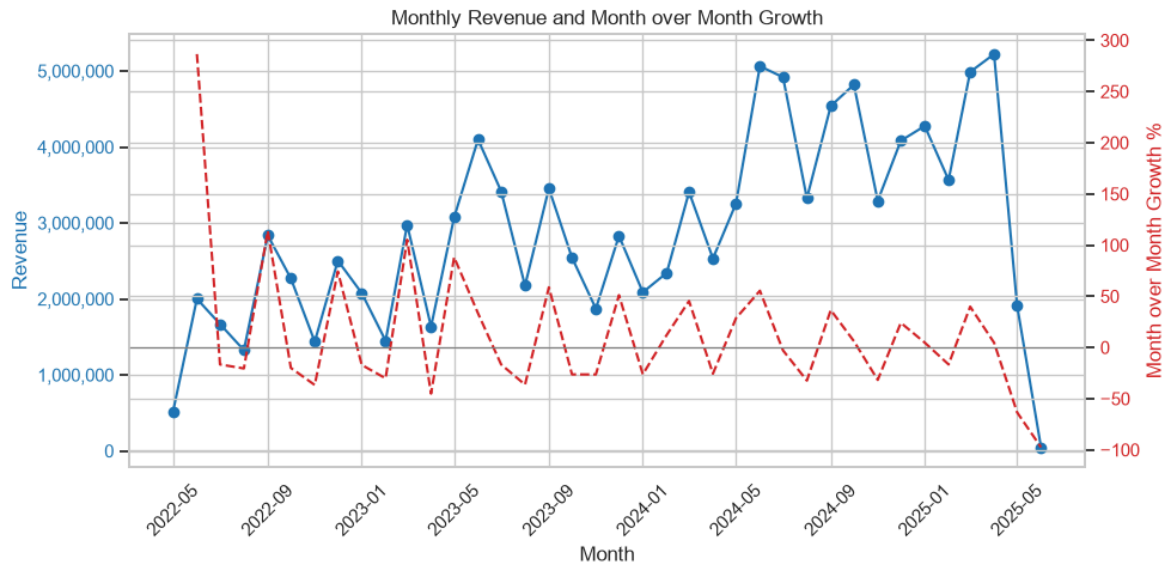
	revenue	mom_growth_pct
count	38.00	37.00
mean	2,890,694.23	12.99
std	1,282,788.56	65.81
min	47,491.55	-97.51
25%	2,020,157.24	-26.35
50%	2,832,907.38	-2.87
75%	3,537,947.26	39.88
max	5,222,758.03	286.54

```
In [7]: fig, ax1 = plt.subplots()

ax1.plot(monthly_revenue["order_month"], monthly_revenue["revenue"], marker="o",
ax1.set_ylabel("Revenue", color="tab:blue")
ax1.tick_params(axis="y", labelcolor="tab:blue")
ax1.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x:,.0f}"))
ax1.set_xlabel("Month")
plt.xticks(rotation=45)

ax2 = ax1.twinx()
ax2.plot(monthly_revenue["order_month"], monthly_revenue["mom_growth_pct"], color="tab:red")
ax2.set_ylabel("Month over Month Growth %", color="tab:red")
ax2.tick_params(axis="y", labelcolor="tab:red")
ax2.axhline(0, color="gray", linewidth=0.8)

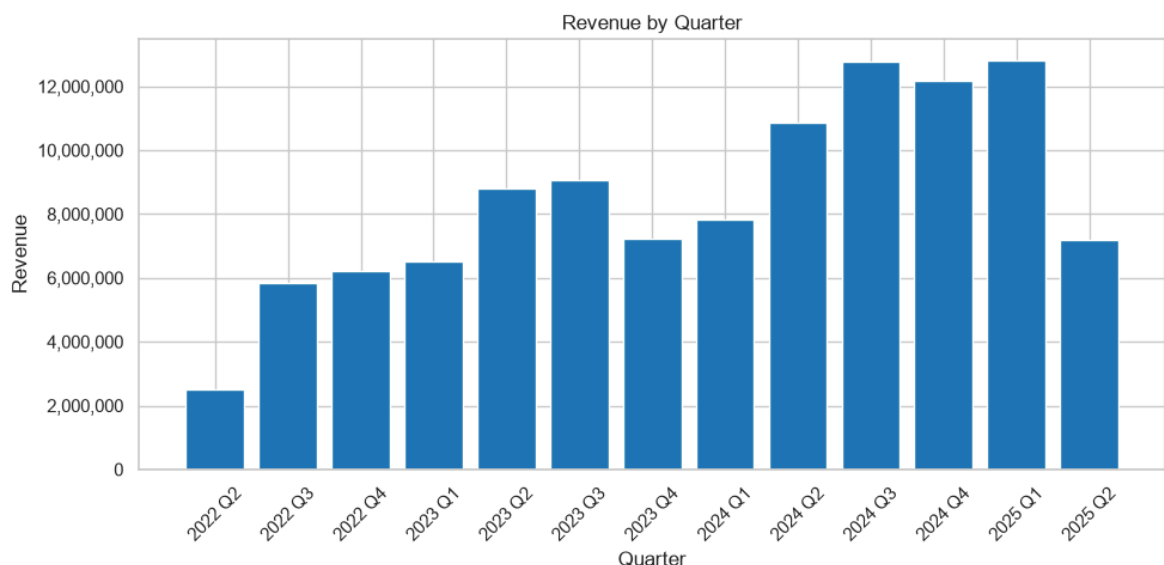
plt.title("Monthly Revenue and Month over Month Growth")
fig.tight_layout()
plt.show()
```



The blue line is total monthly revenue and the red dashed line is the percent change from the prior month, plotted on its own axis since the two are on very different scales. Look for whether growth swings are seasonal (repeating around the same months each year) or one off. Sharp single month spikes followed by a drop back to trend usually point to a promotion or a large one time order rather than a real change in demand.

5. Quarterly Revenue

```
In [8]: fig, ax = plt.subplots()
labels = quarterly_revenue["year"].astype(str) + " Q" + quarterly_revenue["quarter"]
ax.bar(labels, quarterly_revenue["revenue"], color="tab:blue")
ax.set_ylabel("Revenue")
ax.set_xlabel("Quarter")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x:,.0f}"))
plt.xticks(rotation=45)
plt.title("Revenue by Quarter")
plt.tight_layout()
plt.show()
```

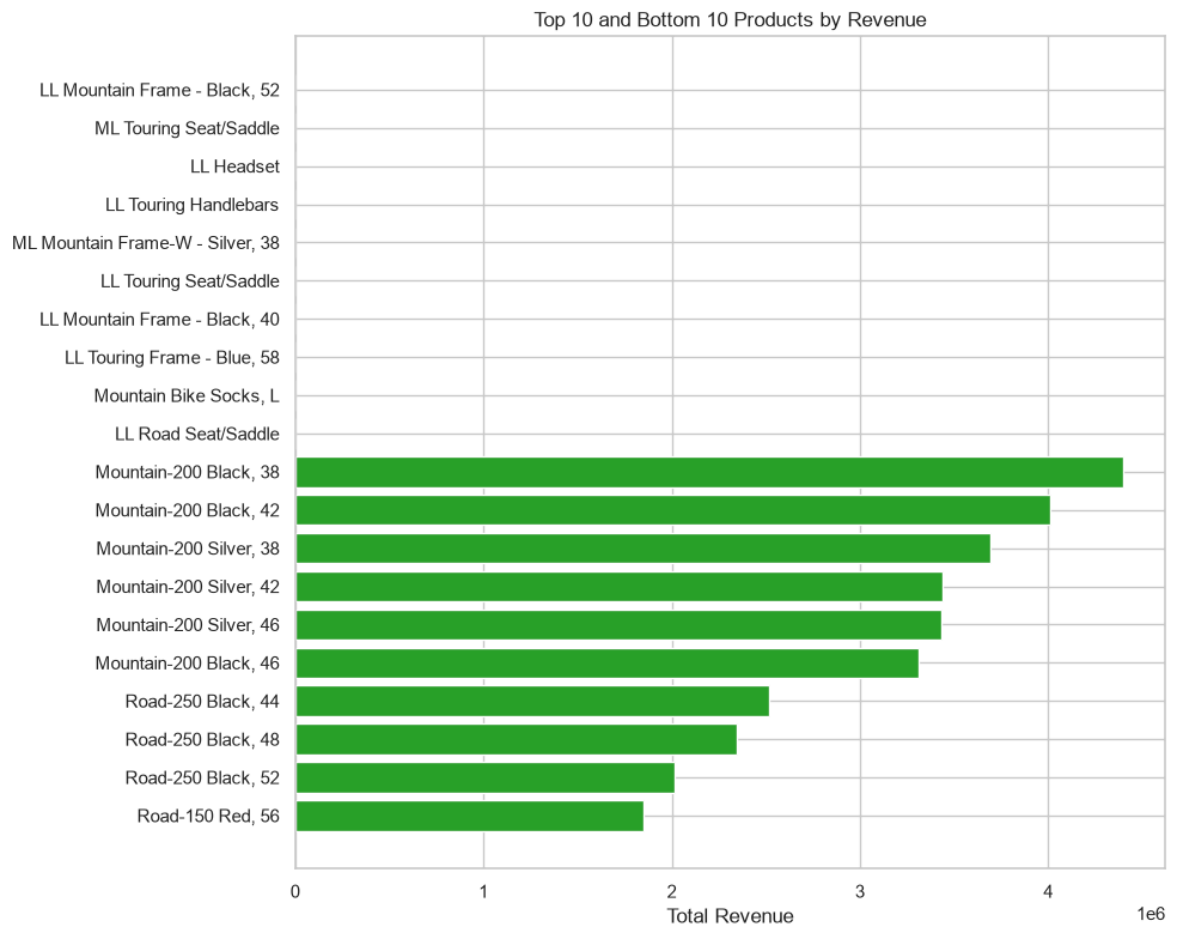


Quarterly view smooths out the month to month noise from the chart above and makes longer term direction easier to see. Compare this against the monthly chart, a

quarter that looks flat here can still contain a volatile month underneath it.

6. Top and Bottom Products by Revenue

```
In [9]: fig, ax = plt.subplots(figsize=(10, 8))
colors = top_bottom_products["rank_type"].map({"Top 10": "tab:green", "Bottom 10": "tab:red"})
ax.barh(top_bottom_products["product_name"], top_bottom_products["total_revenue"], color=colors)
ax.set_xlabel("Total Revenue")
ax.set_title("Top 10 and Bottom 10 Products by Revenue")
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```



```
In [21]: top_bottom_products[["product_name", "rank_type", "total_revenue"]]
top10 = top_bottom_products[top_bottom_products["rank_type"] == "Top 10"]
bottom10 = top_bottom_products[top_bottom_products["rank_type"] == "Bottom 10"]
ratio = top10["total_revenue"].max() / bottom10["total_revenue"].min()
print(f"Top product generates {ratio:.0f}x more revenue than the lowest performing product")
```

Top product generates 27046x more revenue than the lowest performing product

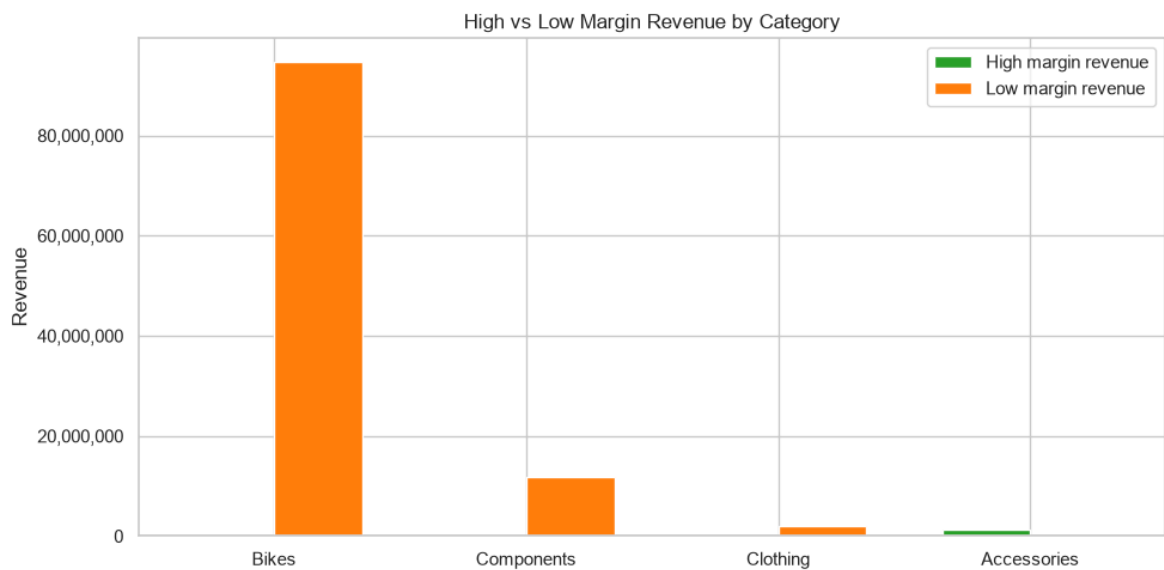
Green bars are the ten highest revenue products, red bars are the ten lowest among products that still have at least one sale. A large gap between the shortest green bar and the longest red bar tells you how concentrated revenue is in a small set of products. If several bottom products have revenue close to zero, check whether they are near end of life or discontinued before treating them as underperformers.

7. Category Performance

```
In [10]: fig, ax = plt.subplots()
x = np.arange(len(category_performance))
width = 0.35

ax.bar(x - width/2, category_performance["high_margin_revenue"], width, label="H
ax.bar(x + width/2, category_performance["low_margin_revenue"], width, label="Lo

ax.set_xticks(x)
ax.set_xticklabels(category_performance["category_name"])
ax.set_ylabel("Revenue")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x:,.0f}"))
ax.set_title("High vs Low Margin Revenue by Category")
ax.legend()
plt.tight_layout()
plt.show()
```

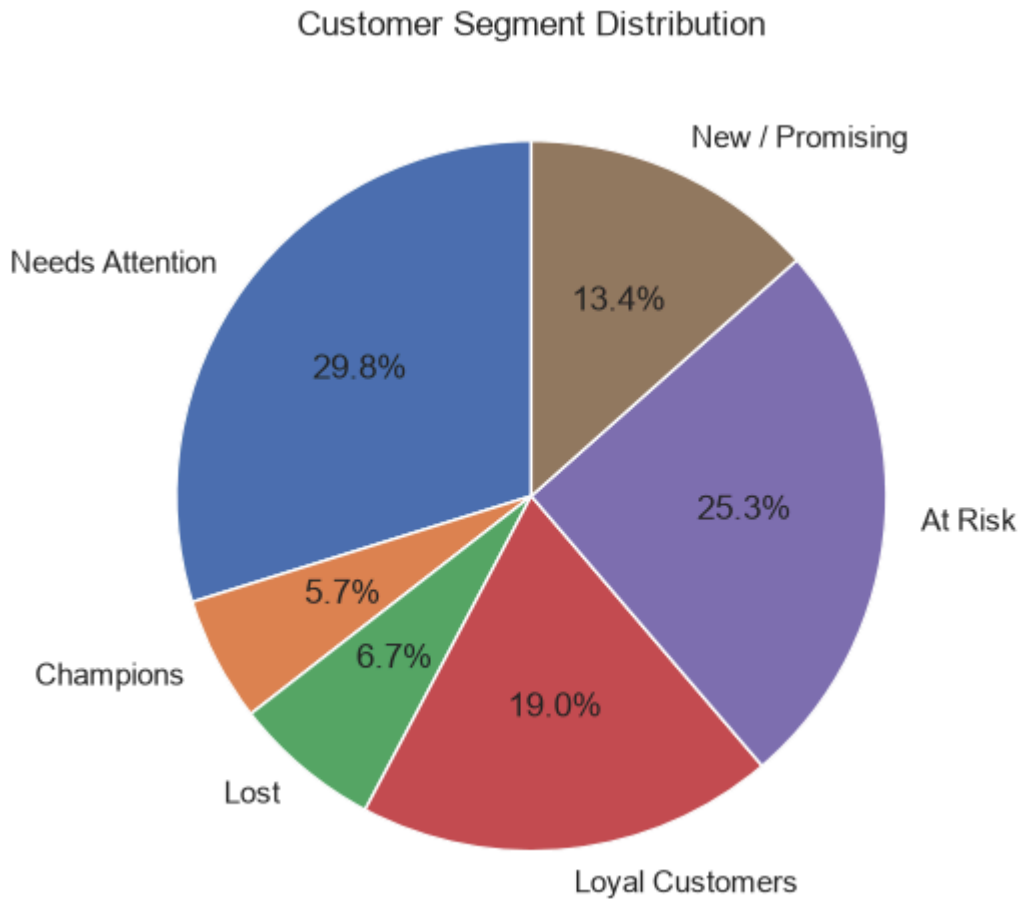


<p>High margin revenue is split from low margin revenue using the 40 percent profit margin threshold set in `analytics.product_performance`. A category with high total revenue but mostly low margin bars is generating volume without much profit behind it, worth flagging separately from a category that is smaller but mostly high margin.</p>

8. Customer Segments

```
In [24]: fig, ax = plt.subplots()
ax.pie(customer_segments["customer_count"], labels=customer_segments["segment"],
ax.set_title("Customer Segment Distribution")
plt.tight_layout()
plt.show()

# Section 8, customer segments
print(customer_segments.sort_values("customer_count", ascending=False))
```

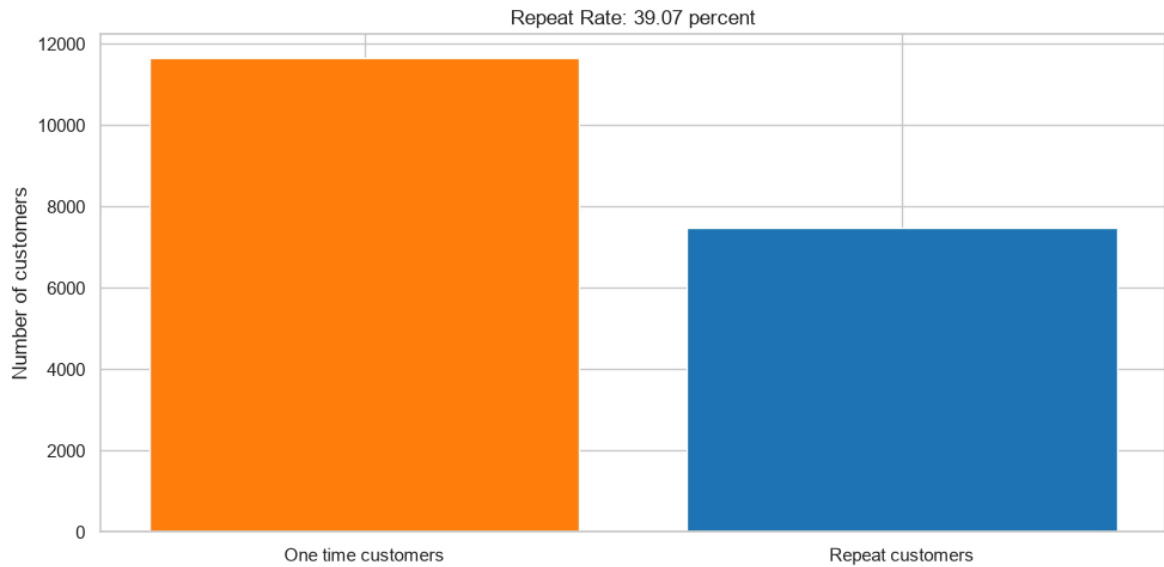


	segment	customer_count
0	Needs Attention	5702
4	At Risk	4837
3	Loyal Customers	3636
5	New / Promising	2569
2	Lost	1288
1	Champions	1087

These segments come from the RFM quartile logic in `analytics.customer_segments`. Champions and Loyal Customers are the smaller premium slices, Needs Attention is typically the largest catch all bucket since most customers land in the middle of the recency and monetary quartiles rather than at the extremes. This matches what was already confirmed during the Stage 3 sanity check, where the segment counts summed exactly to the active customer total.

9. Repeat vs One Time Customers

```
In [12]: fig, ax = plt.subplots()
values = [repeat_customers["one_time_customers"][0], repeat_customers["repeat_customers"][0]]
labels = ["One time customers", "Repeat customers"]
ax.bar(labels, values, color=["tab:orange", "tab:blue"])
ax.set_ylabel("Number of customers")
ax.set_title(f"Repeat Rate: {repeat_customers['repeat_rate_pct'][0]} percent")
plt.tight_layout()
plt.show()
```

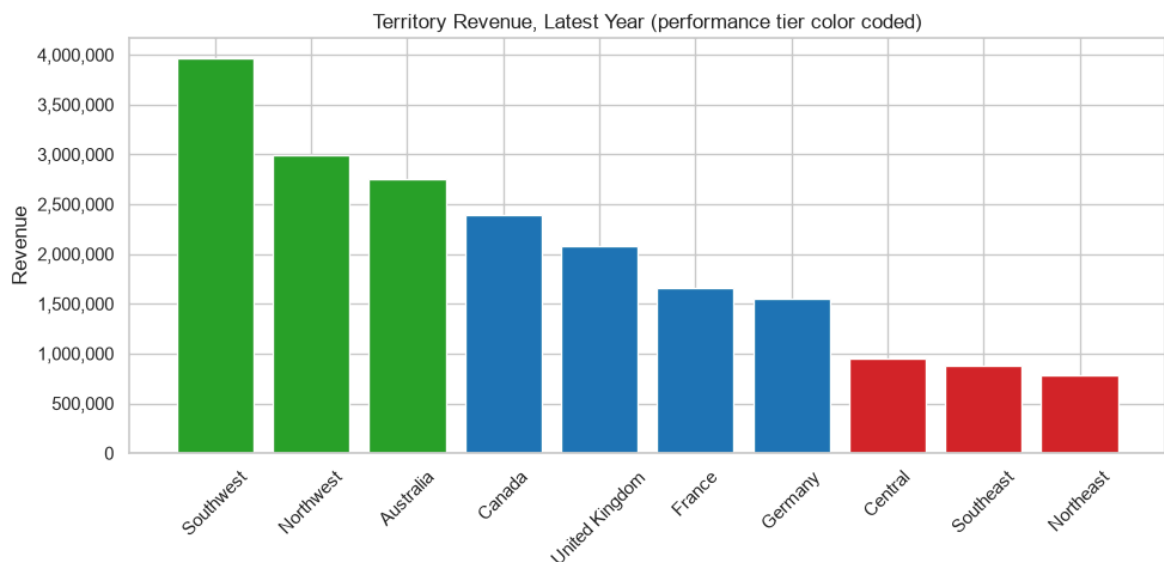


The repeat rate of 39.07 percent was already confirmed as a sane, believable number during the SQL build, neither suspiciously high nor suspiciously low for a retail and wholesale hybrid business like this one.

10. Territory Rankings

```
In [25]: fig, ax = plt.subplots()
tier_colors = territory_rankings["performance_tier"].map({
    "Top Performer": "tab:green",
    "Mid-Tier": "tab:blue",
    "Lowest Performer": "tab:red"
})
ax.bar(territory_rankings["territory_name"], territory_rankings["revenue"], color=tier_colors)
ax.set_ylabel("Revenue")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x:,.0f}"))
ax.set_title(f"Territory Revenue, Latest Year (performance tier color coded)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Section 10, territory
print(territory_rankings[["territory_name", "revenue", "yoy_growth_pct", "performance_tier"]])
```



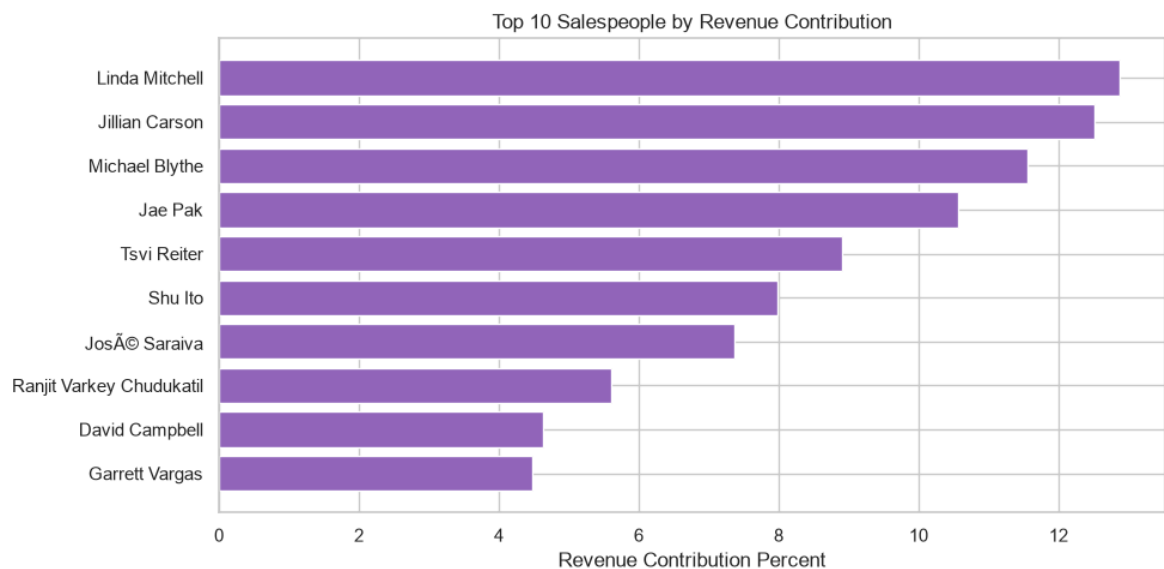
	territory_name	revenue	yoy_growth_pct	performance_tier
0	Southwest	3,966,506.31	-56.52	Top Performer
1	Northwest	2,994,492.67	-50.24	Top Performer
2	Australia	2,751,945.73	-35.19	Top Performer
3	Canada	2,389,120.21	-61.66	Mid-Tier
4	United Kingdom	2,084,356.83	-42.76	Mid-Tier
5	France	1,664,041.51	-56.40	Mid-Tier
6	Germany	1,548,206.90	-39.76	Mid-Tier
7	Central	955,864.52	-68.08	Lowest Performer
8	Southeast	875,603.83	-63.52	Lowest Performer
9	Northeast	778,377.70	-70.42	Lowest Performer

Bars are colored by the performance tier computed in Stage 4, green for top performers, red for the lowest, blue for everything in the middle. Southwest was confirmed earlier as the top revenue territory overall, check whether it is also the top performer in this latest year specifically or whether a different territory has overtaken it recently.

11. Employee Performance

```
In [14]: top_employees = employee_performance.head(10)

fig, ax = plt.subplots()
ax.barh(top_employees["full_name"], top_employees["revenue_contribution_pct"], c
ax.set_xlabel("Revenue Contribution Percent")
ax.set_title("Top 10 Salespeople by Revenue Contribution")
ax.invert_yaxis()
plt.tight_layout()
plt.show()
```



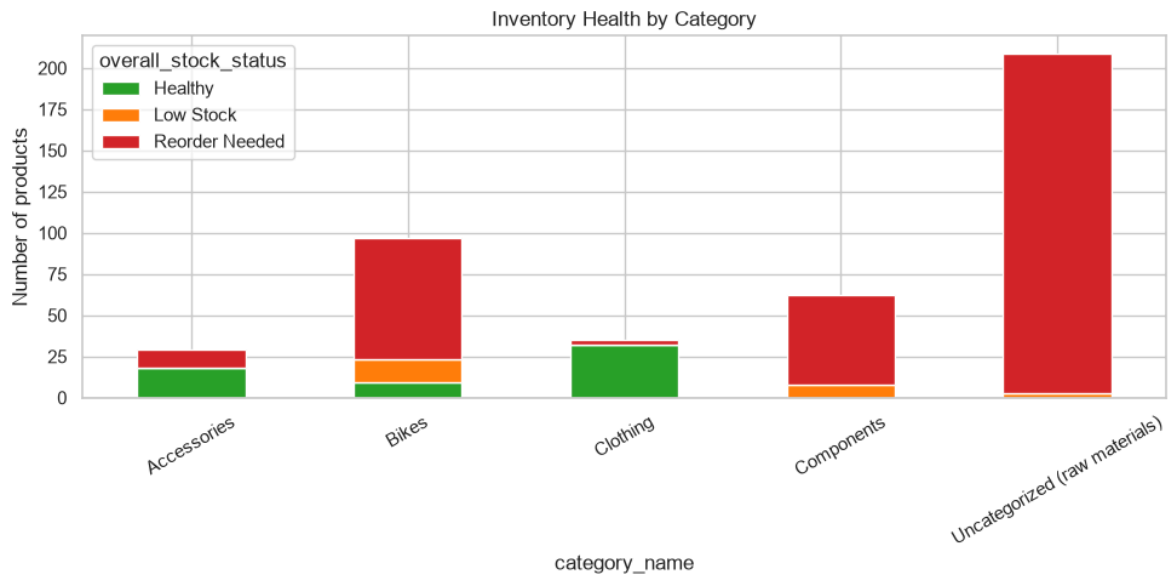
Revenue contribution percent is each salesperson's share of total company revenue, not their individual revenue in isolation, so the bars should feel proportionate to company size. Cross reference against the `quota_status` and `performance_vs_team` columns in the underlying `employee_performance` dataframe, a high revenue contribution paired with Below Quota status usually means their quota was set high relative to their territory rather than a performance issue.

12. Inventory Health by Category

```
In [15]: pivot = inventory_health.assign(
    category_name=inventory_health["category_name"].fillna("Uncategorized (raw m
).groupby(["category_name", "overall_stock_status"]).size().unstack(fill_value=0

pivot = pivot.reindex(columns=["Healthy", "Low Stock", "Reorder Needed"], fill_v

fig, ax = plt.subplots()
pivot.plot(kind="bar", stacked=True, ax=ax, color=["tab:green", "tab:orange", "t
ax.set_ylabel("Number of products")
ax.set_title("Inventory Health by Category")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```



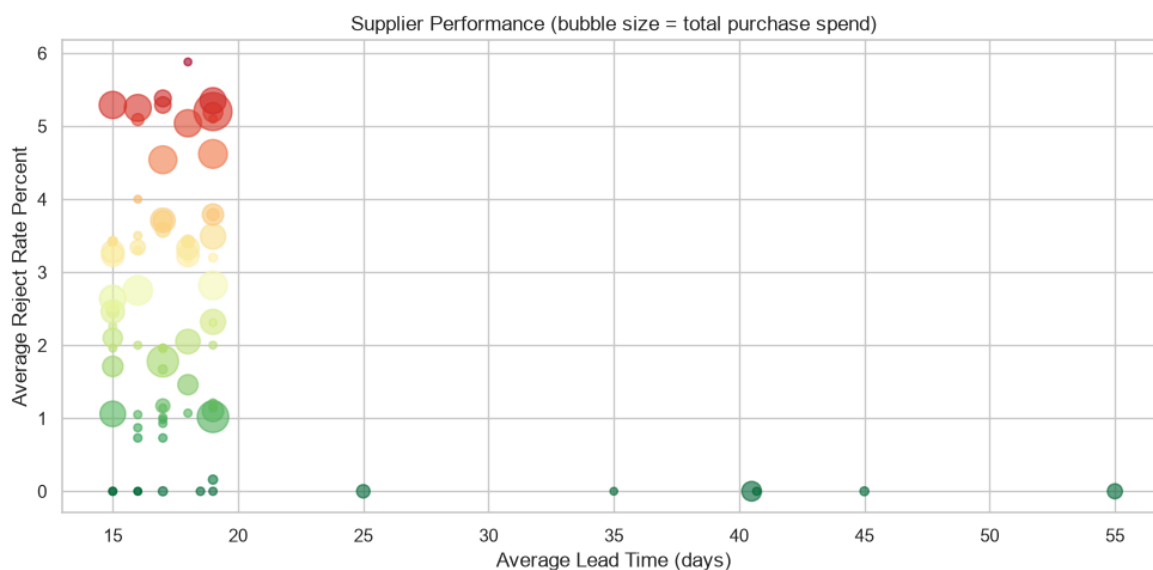
This matches the breakdown already checked by hand during the SQL stage. The Uncategorized bar is raw materials and manufacturing components with no retail category, and it makes up the largest single reorder needed group. Bikes shows a high reorder share too, but that was traced back to tight safety stock margins relative to component categories (a 25 unit gap between reorder point and safety stock level), not a data or logic problem.

13. Supplier Performance: Lead Time vs Reject Rate

```
In [26]: fig, ax = plt.subplots()
sizes = (supplier_performance["total_purchase_spend"] / supplier_performance["to

scatter = ax.scatter(
    supplier_performance["avg_lead_time_days"],
    supplier_performance["avg_reject_rate_pct"],
    s=sizes,
    alpha=0.6,
    c=supplier_performance["avg_reject_rate_pct"],
    cmap="RdYlGn_r"
)
ax.set_xlabel("Average Lead Time (days)")
ax.set_ylabel("Average Reject Rate Percent")
ax.set_title("Supplier Performance (bubble size = total purchase spend)")
plt.tight_layout()
plt.show()
```

```
# Section 13, supplier performance
print(supplier_performance[["vendor_name", "avg_lead_time_days", "avg_reject_rat
```



	vendor_name	avg_lead_time_days	avg_reject_rate_pct	\
85	International	18.00	5.88	
84	Anderson's Custom Bikes	17.00	5.38	
83	Signature Cycles	19.00	5.35	
81	Crowley Sport	15.00	5.29	
82	Capital Road Cycles	17.00	5.29	

	total_purchase_spend
85	7,295.12
84	746,031.82
83	2,023,560.00
81	2,237,800.95
82	697,140.68

Each bubble is a vendor. Bubble size reflects how much total spend goes through that vendor, so a large bubble in the bottom left is a high volume vendor that is also fast and reliable, a large bubble in the top right is a high volume vendor that is slow and error prone, worth prioritizing for review since spend concentration makes their reliability more consequential than a small vendor with the same reject rate.

14. Purchasing Trends

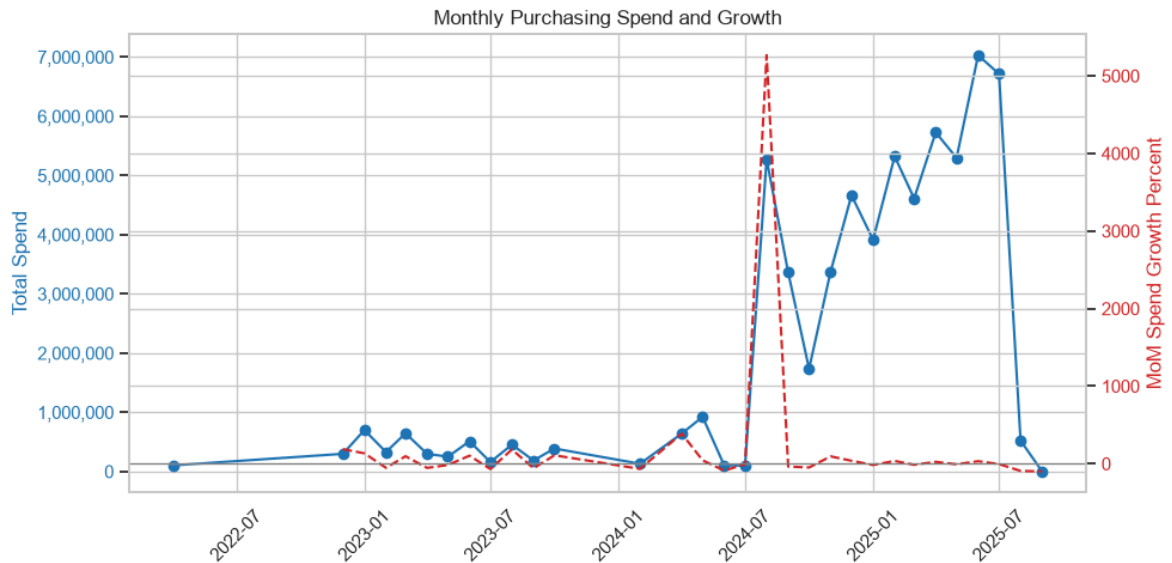
```
In [17]: fig, ax1 = plt.subplots()

ax1.plot(purchasing_trends["order_month"], purchasing_trends["total_spend"], mar
ax1.set_ylabel("Total Spend", color="tab:blue")
ax1.tick_params(axis="y", labelcolor="tab:blue")
ax1.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{x:,.0f}"))
plt.xticks(rotation=45)

ax2 = ax1.twinx()
ax2.plot(purchasing_trends["order_month"], purchasing_trends["mom_spend_growth_p
ax2.set_ylabel("MoM Spend Growth Percent", color="tab:red")
ax2.tick_params(axis="y", labelcolor="tab:red")
ax2.axhline(0, color="gray", linewidth=0.8)

plt.title("Monthly Purchasing Spend and Growth")
```

```
fig.tight_layout()
plt.show()
```



Compare the shape of this chart against the monthly revenue chart from section 4. If purchasing spend rises well ahead of revenue in the same months, that usually means inventory is being built up in anticipation of future sales rather than reacting to current demand, which is a normal pattern for a manufacturing business but worth noting in a writeup.

15. Summary

This dashboard reads exclusively from the `analytics` schema, so every chart above reflects the layered pipeline built in Stages 1 through 4 rather than any fresh computation on raw tables. Re-running Stage 4 in Postgres and then re-running this notebook end to end is enough to refresh every chart with new data.